**Student name:** _____**Student ID:** _____

**Cairo University**
**Faculty of Computers and Artificial Intelligence**
**Subject:** Data Structures
**Subject Code:** SCS214
**Examiner(s):** Dr.Cherry Ahmed/ Dr.Sabah Sayed

**Mid-term exam**
**Semester:** 2nd
**Date:** 30/3/2024
**Duration:** 1 hour

_____
**20**

## Question 1: Sorting: [6 marks]

**A.** How many **key comparisons** and **shift operations** are performed if **Insertion Sort algorithm** is chosen to sort the following array of integers in ascending order. Count only shifts indicated by the statement: a[ j ] = a[ j – 1 ]. Justify your answer by showing all steps (show the array with each pass). **(4 pts)**

| | | | | | |
|---|---|---|---|---|---|
| **6** | **8** | **4** | **7** | **3** | **1** |

**Answer: Comparisons: 14, Shifts: 12** **(1 pts, 0.5 each)**
**(0.75 pt) for each array update**

| | | | | | |
|---|---|---|---|---|---|
| **4** | **6** | **8** | **7** | **3** | **1** |

| | | | | | |
|---|---|---|---|---|---|
| **4** | **6** | **7** | **8** | **3** | **1** |

| | | | | | |
|---|---|---|---|---|---|
| **3** | **4** | **6** | **7** | **8** | **1** |

| | | | | | |
|---|---|---|---|---|---|
| **1** | **3** | **4** | **6** | **7** | **8** |

**B.** For an ascendingly sorted array, the number of moves/assignments performed by selection sort in big-O notation is _____**O(N)** (0.5 pt)____, while for bubble sort the number of moves/assignments in big-O notation would be _____ **O(1)**_____. **(0.5 each)**

**C.** For Mergesort, the space complexity in Big-O is _____ **O(N)**_____, while for quicksort, it is _____ **O(1)**_____ **(0.5 each)**

## Question 2: Linked Lists: [ 8 marks]

For the linked list declaration shown where only a head pointer is kept, answer parts (A) & (B):

**A.** Implement the member function **deleteMiddle** which deletes the node at the middle position of the list. If the list has an even number of nodes, only the second of the 2 middles should be deleted. Handle all special cases. **(5 pts)**

**Example 1:**

***myLst*** has elements: ***20->35->4->10***
After the call: ***myLst.deleteMiddle()***
***myLst*** has elements: ***20->35->10***

**Example 2:**

***myLst*** has elements: ***7->5->4***
After the call: ***myLst.deleteMiddle()***
***myLst*** has elements: ***7->4***

```
template<class T>
class Node
{
 public:
   T       info;
   Node* next;
   Node() {next = 0;}
   Node(T, Node* in=0);
};
```

```
template<class T>
class SLL
{
  Node<T> *head;
public:
  SLL(){ head = 0;}
  void insert(T);
  void deleteMiddle();
};
```

```
void SLL::delMiddle()

{   //to handle special case of empty or 1-node list (1 pt)

    if(!head  || !(head->next))

    {

        delete head;

        head = 0;

        return;

    }
    //to count nodes in list (1 pt)

    int cnt = 0;

    for (Node<T>* current = head; current != 0;current = current->next,cnt++);

    //to get pointer before deleted node (2 pt)
    int half = cnt/2;

    Node<T>* prev = head, *current = head->next;

    for (cnt=0; cnt<(half-1);cnt++)

    {

        prev = current;

        current = current->next;

    }
    //actual delete (1 pt)

    prev->next = current->next;

    delete current;

}
```

**B.** What is the big-O complexity of the following code based on your above implementation of **deleteMiddle()** for a given list myLst of **size n**? O(N) **(1 pt)**

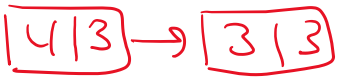
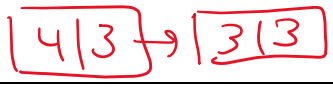
```
for(int i=0; i<10; i++)
  myLst.deleteMiddle();
```

**C.** Suppose we have a pointer to a node (current) in a singly linked list that is guaranteed not to be the last node in the list. We do not have pointers to any other nodes (except by following links). Write an O(1) C++ statements that remove the value stored in such a node from the linked list, maintaining the integrity of the linked list. (Hint: Involve the next node.) **(2 pts)**

```
void delCurr(Node<T>* current)
{
    current->info = current->next->info; (0.5 pt)
    IntNode* del = current->next; (0.5 pt)
    current->next = del->next; (0.5 pt)
    delete del; (0.5 pt)
}
```

## Question 3: Stacks & Queues: [6 marks]

**A.** Draw a priority queue when executing the following sequence considering the priority 1 is the highest priority.
Draw a different queue for each update (an enqueue or a dequeue), showing clearly where the front of the queue: **(2 pts)**

| | |
|---|---|
| (1) enqueue(4,3) |  |
| (2) enqueue(7,1); |  |
| (3) enqueue(3,3); |  |
| (4) dequeue(); |  |
| (5) enqueue(6,2); |  |
| (6) dequeue(); |  |

**B.** Assuming a circular-array-based queue with **first** and **last** indices, and an array of size **SIZE**, implement the member function **int getNumElems()** that returns number of elements currently in the queue. **(2 pts)**

```
int Queue::getNumElems()
{
    if(last >= first)
        return last - first + 1;   (1 pt)
    else
        return SIZE - first + last + 1;   (1 pt)
}
```

**C.** Change the following mathematical expression in infix notation to postfix notation, then evaluate it using a stack. Show the stack content with every update. **(2 pts)**

**7 + 8 * 3 / 4**

**Postfix notation:** 7 8 3 * 4 / + (0.75 pt)

**Show Stack while evaluating the expression:** (1.25 pt)